

Hierarchical Credit Default Prediction

A Two-Stage Cascaded Machine Learning Pipeline

Abstract

This report describes a hierarchical, two-stage machine learning pipeline for credit default prediction. Rather than training a single multi-class classifier, the system decomposes the problem into two binary decisions: first identifying whether a borrower will default, then classifying confirmed defaults as mild or severe. Five models compete at each stage, selected by PR-AUC. Threshold calibration uses F1-optimal search over pooled out-of-fold predictions from stratified 5-fold cross-validation, removing test-set leakage from the calibration step entirely. On a held-out test set of 646 observations, the pipeline achieves 88.5% overall accuracy and a macro F1 of 0.45. No-default prediction is strong ($F1 = 0.94$); performance on the two minority default classes reflects the structural difficulty of the task under severe class imbalance (22 mild defaults, 13 severe defaults in the test set).

Contents

1	Introduction and Motivation	1
2	Data and Preprocessing	1
2.1	Dataset Overview	1
2.2	Numeric Coercion	1
2.3	Missing Value Imputation via KNN	2
3	Pipeline Architecture	2
3.1	Hierarchical Design Rationale	2
3.2	Stage 1: Default Detection	3
3.2.1	SHAP-Based Feature Selection	3
3.2.2	Class Imbalance Handling	3
3.3	Stage 2: Severity Classification	4
4	Model Zoo	4
5	Focal Loss in XGBoost	5

5.1	Motivation	5
5.2	Mathematical Formulation	5
5.3	Custom Objective Implementation	5
6	Threshold Calibration	6
6.1	Why 0.5 is the Wrong Starting Point	6
6.2	Why a Validation Fold is Necessary	6
6.3	Why Stratified Folds are Required	6
6.4	Out-of-Fold Threshold Search	7
6.5	F1-Optimal Threshold Selection	7
7	Results	7
7.1	End-to-End Performance	7
7.2	Per-Class Results	8
7.3	Interpretation	8
7.4	Structural Limitations of the Test Set	9
8	Conclusion	9

1. Introduction and Motivation

Credit default prediction sits at the operational heart of any lending institution. A well-calibrated model does more than separate good borrowers from bad ones. It tells the risk team how bad a potential default might be, which directly informs provisioning, pricing, and portfolio construction decisions.

The conventional approach of training a single multi-class classifier on a three-way outcome (no default, mild default, severe default) runs into two compounding problems in practice. First, the class distribution is severely skewed: non-defaulting borrowers outnumber defaulters by a large margin, and severe defaults are rarer still. A single classifier tends to learn the majority class well while treating both default categories as noise. Second, the two classification decisions – will they default at all, and how badly – are conceptually distinct problems that respond to different features and warrant different modelling strategies.

This report describes a hierarchical, two-stage pipeline designed to address both problems simultaneously. Stage 1 focuses exclusively on separating defaulters from non-defaulters. Stage 2 receives only the confirmed defaulters and decides whether the default is mild or severe. A model zoo of five algorithms competes at each stage, and the winner is chosen on PR-AUC. Decision thresholds at both stages are calibrated on the training data via stratified out-of-fold cross-validation, keeping the test set clean for honest final evaluation.

2. Data and Preprocessing

2.1. Dataset Overview

The dataset consists of two pre-split files: a training set and a held-out test set of 646 observations. Both are loaded with Latin-1 encoding to handle special characters common in financial data exports. The target variable takes integer values where 0 denotes no default and values of 1 or above represent increasing degrees of default severity. The test set contains 611 non-defaulting borrowers, 22 mild defaults, and 13 severe defaults, reflecting a pronounced real-world class imbalance.

2.2. Numeric Coercion

Two financial ratio columns contained non-numeric entries, most likely the result of upstream data quality issues or division-by-zero artefacts recorded as strings: the account receivables to account payables ratio, and the long-term liabilities to total

liabilities ratio. Both were coerced to numeric values using `pd.to_numeric` with `errors='coerce'`, converting non-parseable entries to NaN for subsequent imputation rather than silently dropping rows.

2.3. Missing Value Imputation via KNN

Missing values are imputed using k -Nearest Neighbours imputation with $k = 5$. A primary imputer is fitted exclusively on the full training set and applied to the test set using those same fitted parameters, preventing any leakage of test-set distributional information.

A second, separate imputer is fitted on the defaulters-only subset of the training data before Stage 2 processing. This distinction matters because defaulting companies have a meaningfully different financial profile from the general population: their liquidity, leverage, and coverage ratios occupy a different region of the feature space. Imputing a missing value for a defaulter using neighbours drawn from the full population (which is 94% non-defaulters) would introduce systematic bias by pulling imputed values toward non-defaulter norms. The dedicated Stage 2 imputer finds neighbours exclusively within the sub-population of confirmed defaulters.

KNN imputation is preferable to mean or median imputation in both cases because financial ratios tend to co-vary: a company with low liquidity ratios tends also to show elevated leverage ratios. KNN preserves these inter-feature relationships by filling each missing value using the observed values of the five most similar complete records.

3. Pipeline Architecture

3.1. Hierarchical Design Rationale

Decomposing the three-class problem into two sequential binary decisions has both statistical and practical justifications. Statistically, each binary problem is better-conditioned: the class ratio is easier to manage, and the decision boundary is simpler. Practically, the two decisions map onto distinct business actions. A Stage 1 default flag triggers credit review, while a Stage 2 severity flag triggers provisioning and escalation. The two models can be maintained, monitored, and retrained independently.

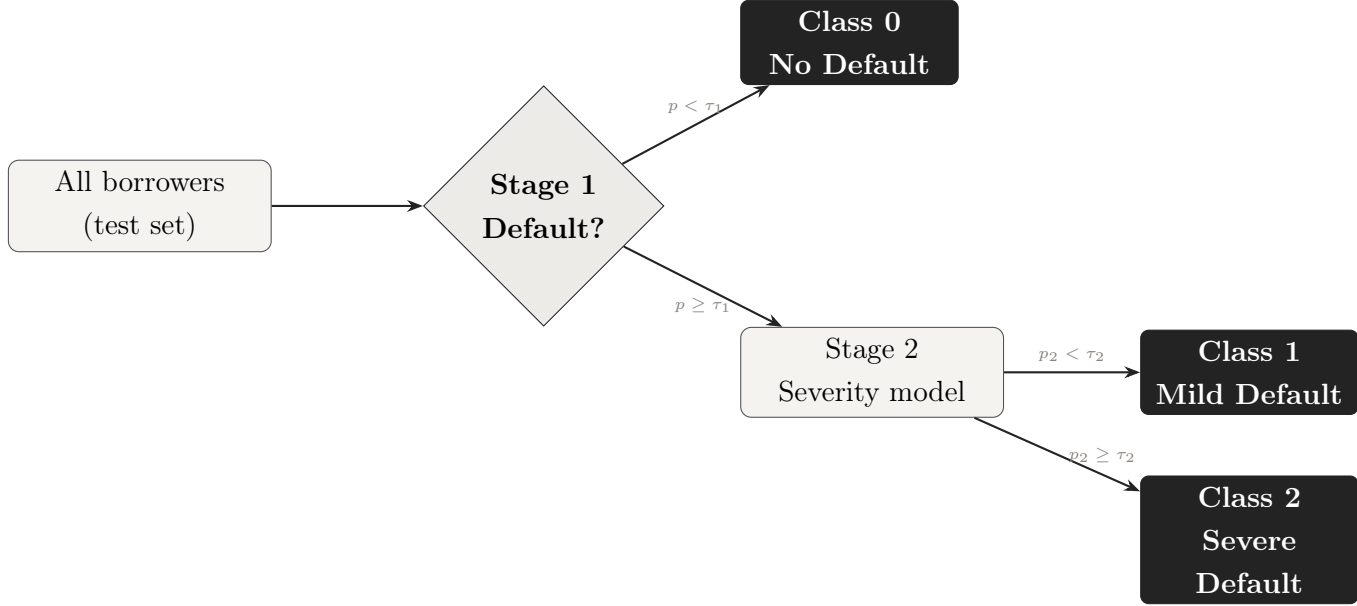


Figure 1: Decision flow of the hierarchical two-stage pipeline. Thresholds τ_1 and τ_2 are selected by F1-optimal search over pooled out-of-fold predictions from stratified 5-fold cross-validation.

3.2. Stage 1: Default Detection

The binary target for Stage 1 is constructed by thresholding the original label: any borrower with a target value above zero is labelled as a defaulter. The stage then proceeds through SHAP feature selection, model training across five algorithms, and threshold calibration via stratified out-of-fold cross-validation.

3.2.1. SHAP-Based Feature Selection

Rather than feeding all available features into the competing models, a preliminary XGBoost classifier is trained on the full feature set and SHAP (SHapley Additive exPlanations) values are computed for each training sample. The mean absolute SHAP value across samples is used as a feature importance score, and the top 20 features are retained for all Stage 1 models. This approach captures non-linear interactions and the importance ranking reflects actual model behaviour rather than marginal statistics.

3.2.2. Class Imbalance Handling

The class imbalance ratio (number of non-defaulters to defaulters) in Stage 1 is 17.82:1. This is computed from the training set and used to set the `scale_pos_weight` hyperparameter in XGBoost and LightGBM, and the class weights argument in CatBoost, Random Forest, and Logistic Regression. This ensures that the minority class contributes proportionally to gradient updates during training.

3.3. Stage 2: Severity Classification

Stage 2 operates only on borrowers already labelled as defaulters. The binary severity target maps original class 1 to mild (0) and all higher classes to severe (1). The class ratio at Stage 2 is 1.71:1, substantially more balanced than Stage 1.

The Stage 1 predicted probability (`s1_prob`) is included as an additional input feature for Stage 2. This passes forward a measure of how confident the first stage was about each defaulter. Borrowers the first model was barely confident about (a probability close to τ_1) may be systematically different from those it was very confident about, and Stage 2 can condition its severity prediction on this signal.

4. Model Zoo

Five models are trained and evaluated at each stage. The model with the highest PR-AUC on the respective test subset is selected as the stage winner. Their principal configurations are summarised in Table 1.

Table 1: *Model configurations used at each stage of the pipeline.*

Model	Key Hyperparameters	Imbalance Strategy
XGBoost	depth 5, $\eta = 0.05$, subsample 0.8, 300 rounds, custom focal loss	<code>scale_pos_weight</code>
LightGBM	depth 5, lr 0.01, 31 leaves, subsample 0.8, 300 rounds	<code>scale_pos_weight</code>
CatBoost	depth 5, lr 0.01, 300 iterations, AUC eval metric	<code>class_weights</code>
Random Forest	300 trees, max depth 10, parallel	<code>balanced</code> weights
Logistic Reg.	ℓ_2 penalty, max 1000 iterations	<code>balanced</code> weights

Gradient-boosted trees (XGBoost, LightGBM, CatBoost) excel on tabular financial data with non-linear feature interactions. Random Forest provides a stable ensemble baseline with lower variance. Logistic Regression, despite its simplicity, often remains competitive on well-preprocessed financial data and serves as a useful sanity check against the more complex models.

5. Focal Loss in XGBoost

5.1. Motivation

The standard approach to class imbalance in boosted trees rescales the loss for minority-class samples via `scale_pos_weight`. This is a uniform rescaling: every positive sample contributes w times as much to the gradient as a negative sample, regardless of how easy or difficult that sample is for the current model to classify. Focal loss, introduced by Lin et al. (2017) in the context of dense object detection, challenges this uniformity. In a well-trained model, the majority of loss is dominated not by hard, informative examples but by the large volume of easily-classified background examples. Even if each one contributes only a small loss individually, their sheer number overwhelms the gradient signal from the difficult examples the model most needs to learn from.

5.2. Mathematical Formulation

The focal loss modifies standard binary cross-entropy by multiplying by a focusing factor:

$$\mathcal{L}_{\text{focal}}(p, y) = -\alpha \cdot (1 - p_t)^\gamma \cdot \log(p_t) \quad (1)$$

where $p_t = p$ if $y = 1$ and $p_t = 1 - p$ if $y = 0$, α is a class-balancing weight, and $\gamma \geq 0$ is the focusing parameter. The pipeline uses $\alpha = 0.25$ and $\gamma = 2.0$. For a well-classified example with $p_t = 0.95$, the scaling factor $(1 - 0.95)^2 = 0.0025$ reduces its gradient contribution by a factor of 400. For a difficult, misclassified example with $p_t = 0.40$, the factor is $(1 - 0.40)^2 = 0.36$, so its contribution is reduced only modestly. The focal loss thereby concentrates the model’s learning budget on hard, uncertain cases.

5.3. Custom Objective Implementation

XGBoost’s built-in objectives do not include focal loss, so it is supplied as a custom objective via the `obj` parameter. This requires providing the first-order gradient and second-order Hessian of the loss with respect to the raw pre-sigmoid prediction:

$$g_i = -\alpha \cdot (y_i - p_i) \cdot (1 - p_i)^\gamma, \quad h_i = \alpha \cdot (1 - p_i)^\gamma \cdot p_i \cdot (1 - p_i) \quad (2)$$

XGBoost uses both for Newton-step updates when constructing each new tree. Supplying an accurate Hessian controls the step size of each update, not just its direction.

Why PR-AUC rather than ROC-AUC?

In highly imbalanced settings, ROC-AUC can appear deceptively high. A model that predicts no default for 99% of cases still achieves a ROC-AUC near 0.5 on a 1% positive-rate dataset, but PR-AUC correctly penalises the near-zero precision. PR-AUC is therefore a more honest measure of whether the model actually recovers the rare positive class.

6. Threshold Calibration

6.1. Why 0.5 is the Wrong Starting Point

Most classifiers default to a decision threshold of 0.5. In credit default prediction this leads to a systematic problem. Because defaulters are rare, the model is trained on data where most labels are zero. Its raw probabilities tend to be anchored low, and many genuine defaulters end up with predicted probabilities of 0.3 or 0.35. A threshold of 0.5 quietly classifies those borrowers as safe, inflating recall on the majority class while missing most of the actual defaults.

6.2. Why a Validation Fold is Necessary

An earlier version of the pipeline selected the threshold directly on the test set by walking the precision-recall curve and stopping at the first point where precision reached 40%. This introduced a subtle but real form of optimistic bias: the threshold was tuned to the test distribution, so performance metrics computed afterwards were evaluated on the same data that had already influenced the threshold choice. The reported figures were slightly more favourable than what the system would achieve on genuinely unseen borrowers. The fix is to move threshold calibration entirely off the test set and onto the training data via cross-validation.

6.3. Why Stratified Folds are Required

With a negative-to-positive class ratio of 17.82:1 in Stage 1, a naive random split would be inadequate. A random 80/20 split of the training data leaves around 40 defaulters in the validation fold. At that scale, the precision-recall curve is extremely noisy – a single misclassified defaulter can shift the curve by several percentage points – and the threshold identified is more a product of sampling variance than a genuine signal. Stratified K-Fold resolves this by enforcing the same class ratio in every fold, so the distribution of positive examples in the validation portion is representative of the full dataset.

6.4. Out-of-Fold Threshold Search

The cross-validation procedure collects out-of-fold predicted probabilities rather than computing a separate threshold per fold and averaging. For each of the five folds, the model is trained on the remaining four folds, and probability estimates are generated for the held-out fold. Because every training sample is held out exactly once, the full training set ends up with a predicted probability from a model that never saw that sample. These probabilities are pooled across all five folds before the threshold search, giving the search an effective validation set the size of the full training data and making the resulting precision-recall curve far more stable than any individual fold.

6.5. F1-Optimal Threshold Selection

Given the pooled out-of-fold probabilities and labels, the threshold τ is selected by sweeping the precision-recall curve and choosing the point that maximises the F1 score:

$$\tau^* = \arg \max_{\tau} F_1(\tau) = \arg \max_{\tau} \frac{2 \cdot P(\tau) \cdot R(\tau)}{P(\tau) + R(\tau)} \quad (3)$$

where $P(\tau)$ and $R(\tau)$ are precision and recall at threshold τ . This replaces the earlier hardcoded precision floor with a criterion derived from observed model behaviour on held-out predictions. The same procedure is applied independently for Stage 2. With a class ratio of 1.71:1 at Stage 2, the F1-optimal threshold sits closer to 0.5, but deriving it from data remains the correct practice.

Why not cost-sensitive threshold selection?

A more principled alternative minimises expected cost: if the cost of a missed default is C_{FN} and the cost of a false alarm is C_{FP} , the optimal threshold satisfies $p(\text{default})/(1 - p(\text{default})) = C_{FP}/C_{FN}$. This encodes the actual business consequences of each error type. In practice, estimating C_{FN} and C_{FP} reliably is non-trivial, particularly for severe defaults which are rare. F1 optimisation serves as a reasonable default when cost data is unavailable.

7. Results

7.1. End-to-End Performance

The pipeline was evaluated on the held-out test set of 646 observations (611 no default, 22 mild default, 13 severe default). Table 2 summarises the top-level results.

Table 2: *End-to-end performance on the held-out test set ($n = 646$).*

Metric	Value
Overall Accuracy	88.5%
Macro F1	0.453
Weighted F1	0.903

7.2. Per-Class Results

Table 3 presents the full classification report. Each row corresponds to one of the three outcome classes; precision, recall, and F1-score are reported alongside the number of test observations in that class.

Table 3: *Per-class classification report on the held-out test set.*

Class	Precision	Recall	F1	Support
No Default	0.97	0.92	0.94	611
Mild Default	0.28	0.23	0.25	22
Severe Default	0.11	0.38	0.17	13
Macro avg	0.45	0.51	0.45	646
Weighted avg	0.93	0.89	0.90	646

7.3. Interpretation

The 88.5% overall accuracy and weighted F1 of 0.90 are inflated by the dominance of the no-default class, which accounts for 94.6% of the test set. Macro F1 of 0.45, which weights each class equally regardless of size, is the honest figure and reflects the genuine difficulty of the task.

The no-default class performs well at an F1 of 0.94, as expected for a well-represented majority class with strong signal in the feature set. The mild default class is the hardest problem for this pipeline: precision of 0.28 and recall of 0.23 produce an F1 of 0.25, meaning the model correctly identifies only around one in four mild defaulters. The severe default class shows a different pattern – recall of 0.38 indicates the model catches a meaningful fraction of severe cases, but precision of 0.11 means that most of the borrowers the model flags as severe are not actually severe. The pipeline is trading precision for recall in the severe class, which may be an acceptable trade-off

given the disproportionate cost of a missed severe default.

The asymmetry between mild and severe performance is worth noting. The mild default class has more observations (22 versus 13) but lower recall, suggesting that mild defaults are inherently harder to distinguish from non-defaults on the available features. Severe defaults are more extreme deviations from the non-default profile and are consequently easier for the model to identify when it does flag something, though the high false-positive rate for severe flags reflects the difficulty of separating severe from mild at Stage 2 with only 35 defaulters in the training subset.

7.4. Structural Limitations of the Test Set

These results should be interpreted with an awareness of the test set composition. With 22 mild defaults and 13 severe defaults, single misclassifications produce large percentage swings in per-class metrics. An improvement from 5 to 6 correct severe-default identifications would shift severe recall from 38% to 46%, a figure that looks substantial but represents one observation. The macro F1 and per-class metrics are directionally informative, but confidence intervals on these estimates would be wide.

8. Conclusion

The hierarchical pipeline addresses the two principal difficulties of credit default modelling – class imbalance and multi-class complexity – by decomposing the problem into two focused binary decisions. On a test set of 646 observations, it achieves 88.5% overall accuracy and a macro F1 of 0.45. The no-default class is classified with high confidence ($F1 = 0.94$). Performance on the minority classes – 0.25 for mild default and 0.17 for severe default – is limited primarily by the small number of defaulters available for training Stage 2 (35 total in the test set) and by the inherent difficulty of distinguishing mild defaults from non-defaults on financial ratio features alone.

Three methodological improvements distinguish this version from the initial implementation: threshold calibration via stratified 5-fold cross-validation with pooled out-of-fold probabilities, eliminating test-set leakage; F1-optimal threshold selection replacing a hardcoded precision floor; and a dedicated Stage 2 imputer fitted on confirmed defaulters to avoid pulling imputed values toward non-defaulter norms.

The most impactful near-term improvement would be cost-sensitive threshold selection, which requires estimating the actual financial cost of a missed default versus an unnecessary review. This would allow the precision-recall trade-off at each stage to be set according to the real operational consequences rather than a symmetric

F1 criterion. Beyond that, severity-specific SHAP feature selection and temporal cross-validation schemes better suited to out-of-time evaluation remain open areas of improvement.